IE498: Machine Learning in Finance

Group Project

Chapter 1: Credit Score

Yashvi Bhatt

Gabriel Reisen

Micaela Perez

Juan Andres Marsano

Introduction

Background and Context

The original dataset contained 26 financial indicators and their corresponding Moody's credit ratings. Our goal was to develop models capable of predicting a company's credit rating based on its financial indicators. Since the ratings were categorical strings and the dataset included minority classes, an early challenge was determining how to best encode these ratings numerically and whether to consolidate them into broader groups. After defining two grouping schemes, we trained several models to predict each target and evaluated the potential and effectiveness of ensembling these models to improve predictive performance. In the next pages we'll explore the groupings, the models we trained, their ensemblings and the result we obtained from them.

Exploratory Data Analysis

In order to implement efficient models for the data, we first have to understand what we are working with and the intricacies of both features and targets. Considering the goal of this project is to optimize classification methods, the exploratory data analysis aims to paint a picture of the nature of the features, how they relate amongst themselves, and how they naturally relate to the targets.

General and Statistical Summary

To get a better sense of the data frame at work, the structure and dimensions of the frame were explored through a preview of the structure including the first 17 features printed, as shown in Figure 1 below, and a call to the shape function which yielded dimensions (1700, 29).

	Sales/Revenues	Gross Margin	EBITDA	EBITDA Margin	Net Income Before Extras	Total Debt	Net Debt	LT Debt	ST Debt	Cash	Free Cash Flow	Total Debt/EBITDA	Net Debt/EBITDA	Total MV	Total Debt/MV	Net Debt/MV	CFO/Debt
0	-0.005496	0.030763	0.018885	0.024515	0.146849	-0.029710	-0.019296	-0.042648	0.049875	-0.133716	0.352997	-0.047694	-0.037474	0.263605	-0.232129	-0.223883	0.101245
1	-0.005496	0.030763	0.088716	0.094733	0.146849	-0.029710	-0.019296	-0.042648	0.049875	-0.133716	0.352997	-0.108776	-0.099211	0.045420	-0.071864	-0.061909	0.101245
2	-0.007045	0.023159	0.088716	0.096440	0.108590	0.039410	0.034268	0.009059	0.250371	0.101315	0.500174	-0.045288	-0.050011	0.060519	-0.019903	-0.024748	0.027638
3	-0.009396	0.028400	0.088716	0.099046	0.146137	0.030071	0.036938	-0.016964	0.356994	-0.052606	0.551592	-0.053866	-0.047559	0.142160	-0.098139	-0.092127	0.042437
4	-0.009009	0.027714	0.088716	0.098611	0.123500	0.024224	0.034445	-0.034132	0.461894	-0.090869	0.753321	-0.059237	-0.049849	0.147989	-0.107808	-0.098904	0.106052

Figure 1: Snippet of Data Frame for the Credit Score Data Set

To ensure that no instances were repeated and that there was no null data, both a count of duplicates and a count of rows with null values was implemented, both yielding a total of 0 for

each and every feature, meaning the data was ready to work with. Seeking to get a first sense of the statistical behavior, the describe function was used to get a statistical summary of each of the numerical features of the frame and presented in Figure 2 below.

```
Basic Statistics for Numerical Features:
       Sales/Revenues  Gross Margin       EBITDA  EBITDA Margin  \
count     1700.000000   1700.000000  1700.000000    1700.000000
mean         0.050378      0.026007     0.068718       0.021074
std          0.161910      0.273768     0.237365       0.189025
min         -0.661715     -0.794722    -0.782254      -0.805153
25%         -0.005693     -0.020028    -0.022640      -0.042771
50%          0.034000      0.003403     0.049482       0.011134
75%          0.083004      0.025595     0.124533       0.060566
max          2.277229      3.202713     3.542425       4.141182

       Net Income Before Extras   Total Debt     Net Debt      LT Debt  \
count               1700.000000  1700.000000  1700.000000  1700.000000
mean                   0.123026     0.822405    -0.419810     1.255168
std                   14.475689    13.317075    28.385702    16.224453
min                 -289.000000    -0.903014  -493.305578    -0.921515
25%                   -0.158478    -0.076316    -0.120725    -0.094767
50%                    0.056627     0.005886    -0.003060    -0.002078
75%                    0.222219     0.136449     0.160251     0.174735
max                  478.280075   281.604237   865.194595   289.388178

           ST Debt         Cash  Free Cash Flow  Total Debt/EBITDA  \
count  1700.000000  1700.000000     1700.000000        1700.000000
mean      3.142797     0.466620       -0.312325           0.731197
std      51.986550     1.859494        8.895136          12.280493
min      -0.997692    -0.990982     -238.750000          -0.910486
25%      -0.337959    -0.195117       -0.527219          -0.134477
50%       0.043092     0.075820       -0.058475          -0.012302
75%       0.649475     0.483113        0.396581           0.141443
max    2038.000000    36.980037      125.785869         256.050232
```

Figure 2: Statistical Description of the first 12 Feature

<u>Target Variable Analysis</u>

In order to get a better understanding of how the two targets behave and appear in the data, their distributions were explored through a pie chart for the Investment Grade (the binary target) and a bar graph for the Rating (multi-class target) as seen in Figure 3 below.

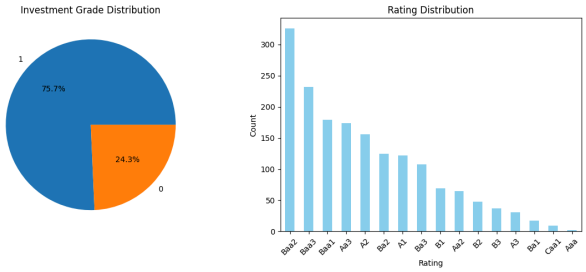


Figure 3: Data Instances Grouped under the different Targets (Investment Grade and Ratings)

Considering that instances gain the attribute of investment grade depending on their moody scale rating, with the threshold being that ratings greater Baa3 earn the title, the bar graph clearly explains the pie chart as the five most frequented ratings are all investment grade.

Key Financial Ratios Distribution

Considering the more traditional financial indicators we plotted histograms for each of them (Figure 4) in order to visualize the distribution of each of them. Based on the plots we were able to identify that the metrics directly related to cash flow, Sales, Gross Margin and EBITDA were better distributed in comparison to the other plotted metrics. In addition the histograms further reinforced our concern that the dataset contained possible outliers.

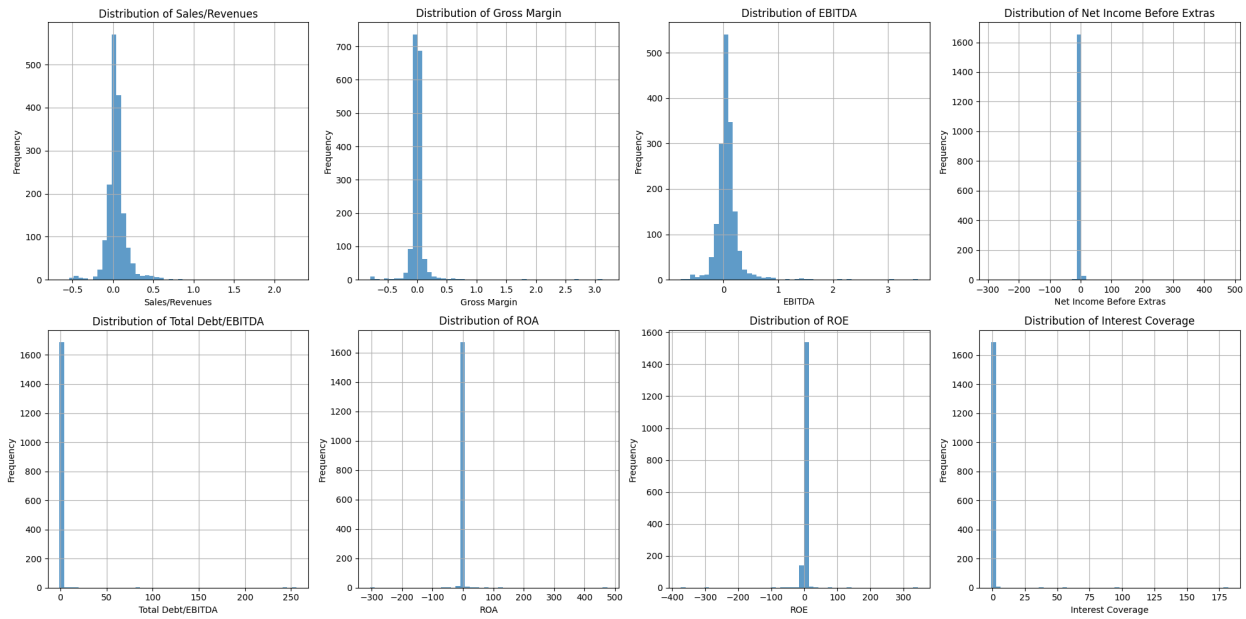


Figure 4: Data Distributions for the selected Key Financial Ratios

Correlation Analysis

From the preview of the data, various features were expected to be correlated as some were a function of others and directly depended on them, such as the Total Debt and Total Debt/MV. To confirm these suspicions, a correlation heat map was generated using a correlation matrix and the results displayed in Figure 5 below.

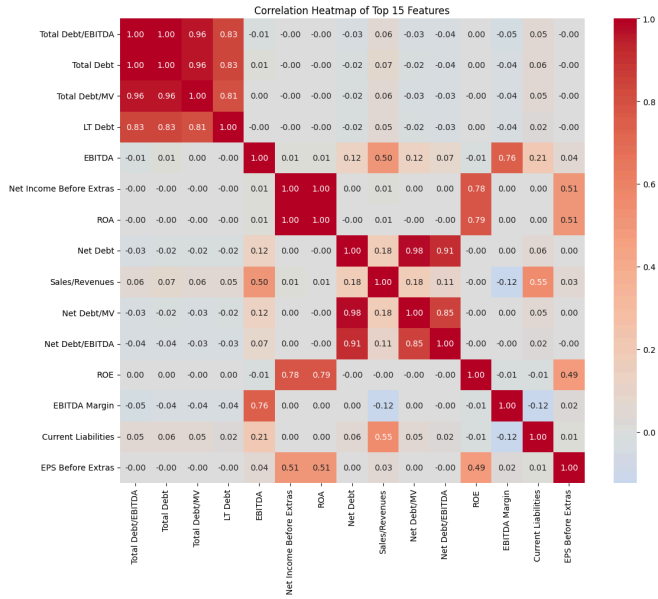


Figure 5: Correlation Heat Map for the Top 15 Features

As hypothesized, there is a set of features with extremely elevated correlation values between one another, thus a red flag was raised and we began exploring possible measures to take regarding these features, these steps were explored and evaluated during data preprocessing.

<u>Feature Distribution by Rating</u>

Seeking to get a better sense of the relationship between some of the key features and the multi-class target, as well as to contrast these relationships for highly correlated features, box plots for each of the classes were generated and presented in Figure 6 below.

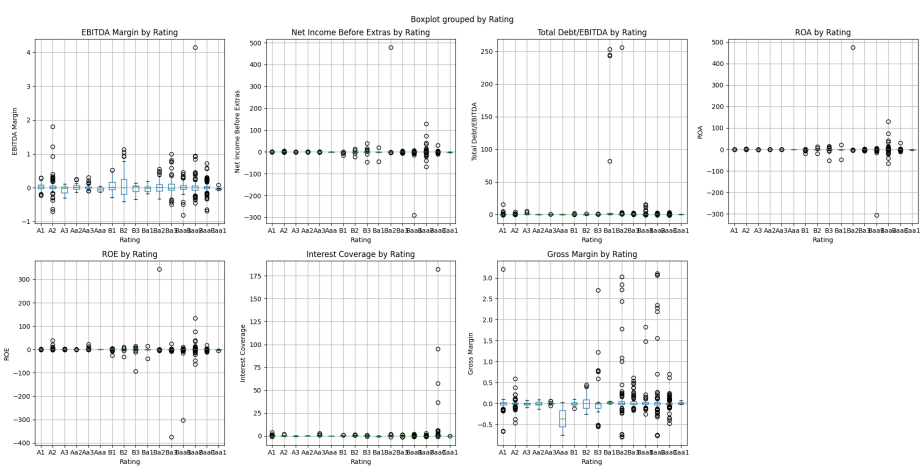


Figure 6: Feature Distribution by Rating

Evidently, this raised a second red flag as the quartiles are nearly indistinguishable from each other and the distributions are defined through the quantity and spread of the outliers. Consequently, methods to deal with the significant amount of outliers were drafted and evaluated to continue with the preprocessing of the data.

Preprocessing/Feature Extraction/Feature Evaluation

Preprocessing

As aforementioned, the data set did not include any duplicates or null values, therefore preprocessing lied solely on the formulation of a more efficient and applicable multi-class target, dealing with outliers, and scaling the data for effective model fits.

Grouping Ratings

Considering the extensive number of classes, which amounted to 16, and the reduced frequencies for some of these, there was an expectation for multi-class models to experience reduced accuracies due to the insufficient signals for each. Therefore, accounting for the fact that the Moody Rating scale has overarching categories ranging from prime to lowest, a function was generated to group the ratings into their overarching classes. Namely, Aaa remained as its own category titled 'Prime", all of the Aa ratings became 'High Grade', A ratings became 'High Medium', Baa ratings became 'Lower Medium', Ba ratings became 'Speculative', B ratings became 'Highly Speculative', Caa ratings became 'Substantial Risk', Ca rating became 'Near Default', and the C was renamed into 'In Default'. Due to the very small size of the "Prime" group of only 2 observations we judged best to merge it with the "High Grade" classification group. The final transformation to this target lies in enforcing these categories as numerical values in order for the models to effectively take them in as targets. One hot encoding was considered, yet, the linearity of assigning numbers 1-6 was considered valuable as it reflects the significance behind each category. To test the effect of the grouping function on the frequencies of each class, the multi-class target bar graph was run again and the results presented in Figure 7 below.

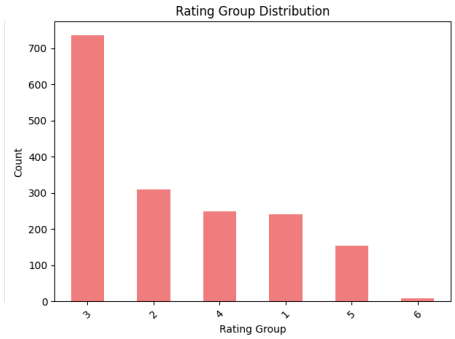


Figure 7: Data Instances Grouped under the Final Multi-Class Target (Rating Group)

Although considered as part of the original groupings due to Moody's rating structure, classes 7 and 8, corresponding to Near Default and In Default respectively, did not appear in the 1700 instances. The figure shows that we effectively met our goal and incremented the frequencies per class, yet, will have to consider the effect the limited occurrences of 'Substantial Risk' signals will have on the accuracy of the multi-class model. To further explore the effect of the function on the multi-class target, outliers were evaluated by re-running the feature distribution box plots and the results presented in Figure 8 below.

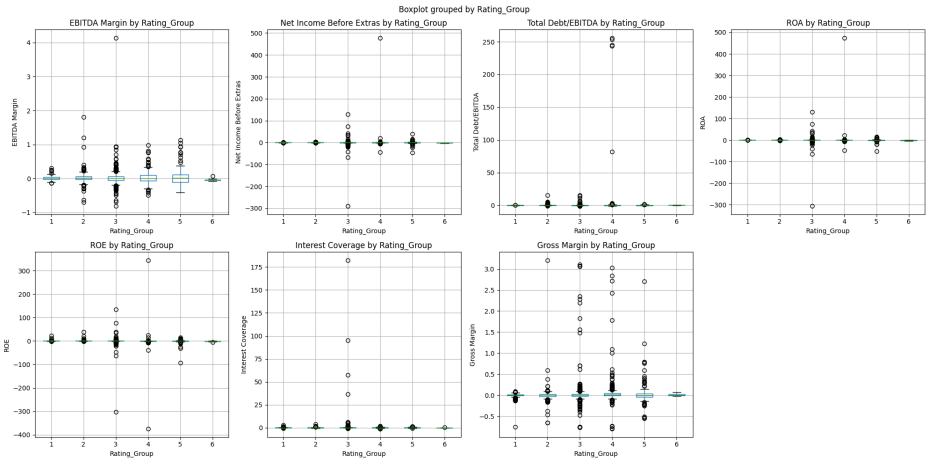


Figure 8: Feature Distribution by Rating Group

<u>Outliers</u>

Seeing as the outliers remained, their importance and contributions were conceptually evaluated to determine how to deal with them. Recognizing the value in these outliers regarding their

potential efficiency when used for classification, we employed a robust scaler. Since this scaler centers on the median and scales by IQR, it is resistant to extreme values and therefore an effective fit for preserving the valuable information the outliers could provide whilst dampening the impact of extremes.

Feature Evaluation

In regards to evaluating the contributions and importance of each of the training features offered, Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), and the Random Forest method "Feature Importance" were considered. Seeking to preserve feature meaning and identity, PCA was discarded as its generation of components diminishing the interpretability of the logic future models and especially because it has a known limitation regarding its effectiveness in classification endeavors. Regarding LDA, the unequal covariances amongst features and the fact that the models we planned to use do not draw straight lines on the feature space while LDA specializes in drawing straight lines through said space, rendered this approach limited. Furthermore, since we opted to explore tree based models such as random forest and ADABoost and these are able to deal with the larger dimensional data well, we opted to evaluate the model on the original features, which would also permit us to look at feature importance.

In addition to considering PCA and LDA we took into consideration how to deal with the highly correlated features, focusing mainly on the ones that showed correlation with each other of >90%. Since we identified that all of these highly correlated features had been derived from other features mainly consisting of ratios between them it felt appropriate to remove one of the ends of each high correlation without risk of losing valuable information since their information would still be contained in the remaining features, the features we removed from the dataset were Total Debt, Total Debt/MV, Net Debt, Net Debt/MV and Net Income Before Extras.

Model Fitting and Individual Evaluation

Once the data was preprocessed and features amended, three models were tested and their hyperparameters tuned to evaluate their performance on the data set using the accuracy metric. Though the F1 Macro and Precision metrics were considered for evaluation, the accuracy metric's stability and interpretability seemed to better fit our needs given the large number of

estimators in categories 1-5, addressing the minority class 6 estimations through the ADABoost model's capability of focusing on minority classes.

All models were integrated into pipelines that first ran the robust scaler and then the model itself to then have the selected hyperparameters tuned and their performance evaluated where each model's hyperparameters were tuned using Grid Search Cross-Validation with five folds. Individual run times for the separate grid searches were recorded to aid in future evaluation of the grid set ups especially considering further and more precise tuning. Considering the limitations of the Google Collab environment, Virtual Studio Code in the personal machines was used to run the model training and the parameter tuning due to better runtime performance, the runs for this are found in the github version of the code and pasted as text onto the main Google Colab file.

Multiple Class Classification Models

For the use of the rating group as a target, the selected classification models were Random Forest, AdaBoost, and Logistic Regression. Random forest was selected due to its stability, its robustness to noise, and as well as its ability to identify feature importance. AdaBoost was selected due its focus in hard to classify cases through its reweighing of mistakes. Logistic regression was considered due to its fast, stable, and simple nature, a perfect counterpart to the more complex models used before and potentially benefiting during the ensembling phase due it counteracting potential overfits from the other models.

Random Forest

The selected hyperparameters for the random forest were the number of estimators, the maximum depth of each tree, the minimum samples required to split an internal node, the minimum samples required at a leaf, and the maximum number of features to consider when looking for the best split. Moreover, to balance computational run time and preserve simplicity, gini impurity was selected as the criterion used to split nodes. The values tested in the grid search as well as the results of the hyperparameter tuning (the best parameters and thus the ones that maximize accuracy) for this model are presented in Table 1.

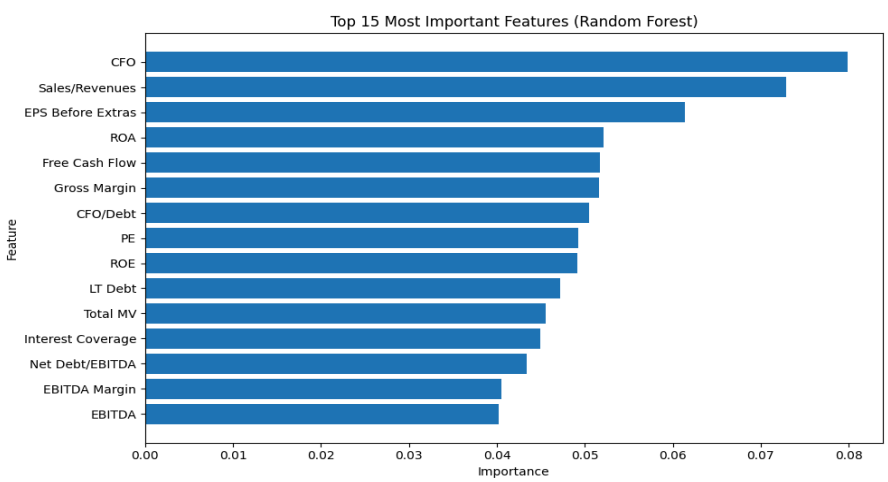
Table 1: Hyperparameter Tuning Values and Results for the Random Forest Classifier Model

Hyperparameter	Tested Value/Methods	Best Value/Method
Number of Estimators	300, 600, 900, 1200	1200
Maximum Tree Depth	None, 8, 12, 16	None
Minimum Samples to Split	2, 5, 10	2
Minimum Samples per Leaf	1, 2, 4	2
Maximum Features Considered	sqrt, log2, 0.5	0.5

Once the best parameters were identified by the grid search, the random forest grid was fit to the training data and then its accuracy was tested, and the best cross-validation accuracy was approximately 0.672 and the runtime 25 minutes 2.5 seconds.

In figure 9 we are able to see which features were the most significant in predicting the credit ratings, with CFO, Sales and EPS Before Extras being the most influential, although we did not see that many variations in the importance of the remaining top 15 features.

Figure 9: Feature Importance as calculated by the Random Forest feature_importances_ method



<u>AdaBoost</u>

In the implementation of the AdaBoostClassifier object, decision stumps (maximum depth of one) were selected to contrast the freedom and increased depth allowed for the random forest,

and the stumps were expected to provide further stability for the general model. The selected hyperparameters for the AdaBoost model were the learning rate, the number of estimators, the maximum depth of each estimator, the minimum samples per leaf, and the class weight was set to None as other values presented high error rates that weakened the reliance on the results of the search. The values tested in this grid search and the results of the tuning for this model are presented in Table 2.

Table 2: Hyperparameter Tuning Values and Results for the AdaBoost Classifier Model

Hyperparameter	Tested Value/Methods	Best Value
Learning Rate	0.01, 0.05, 0.1, 0.2, 0.3	0.3
Number of Estimators	300, 500, 700, 900, 1100	1100
Maximum Depth	1, 2, 3	3
Minimum Samples per Leaf	1, 2, 3, 4, 5	5

Using the best values for the hyperparameters, the AdaBoost models fit and yield a best cross-validation accuracy of approximately 0.554 alongside a 24 min and 59.6 seconds runtime.

<u>Logistic Regression</u>

For the exploration of the logistic regression model as a candidate the selected hyperparameters were the scaler, the L1 Ratio as we set elastic net as the regularization type, the value of C, the class weight, and the optimizer tolerance which serves as the stopping criterion for the solver with smaller values indicating stricter convergence. The values for the grid search for this model are presented in Table 3 alongside the result for the best combination of hyperparameters.

Table 3: Hyperparameter Tuning Values and Results for the Logistic Regression Model

Hyperparameter	Tested Value/Methods	Best Value/Method
Scaler	Robust Scaler, Standard Scaler	Robust Scaler
L1 Ratio	0.0, 0.25, 0.5, 0.75	0.25

Class Weight	None, Balanced	None
Total	10^{-4}, 10^{-5}	10^{-5}

Using the results of the grid search, the logistic model was trained and the best cross-validation accuracy was found to be approximately 0.445 with a runtime close to 46 minutes and 52 seconds.

Binary Classification Models

For the use of the Investment Grade data as the target for the models, simpler classifying models than the ones used for the multiple class target were selected to maintain the balance expected by the final integration of the models into a single object. Therefore, we implemented and tuned the hyperparameters for a Decision Tree Classifier and a Logistic Regression model. Moreover, considering the simplicity of the models and the reduced amount of hyperparameters to test for each model, a larger amount of folds was justified and we opted to use 10. In contrast with the

Decision Tree

As with the random forest model, the hyperparameters tuned for this model were the maximum depth, the minimum sampler per leaf, and the minimum samples to split. Values tested for each of the hyperparameters and the result of the search are presented in Table 4.

Table 4: Hyperparameter Tuning Values and Results for the Decision Tree Classifier Model

Hyperparameter	Tested Value/Methods	Best Value
Maximum Depth	1, 3, 5, 8, 10, 14, 20	14
Minimum Samples per Leaf	1, 5, 10	1
Minimum Samples to Split	2, 10, 20	2

Once the tree grid was fit to the training data and validated, it yielded a best cross-validation score of approximately 0.786 and a runtime of 1.2 seconds.

Logistic Regression

Considering the range of regularization types, we decided to test all of them and feed them the same values for the hyperparameter tuning. In other words, we tested whether Lasso (L1), Ridge (L2), and Elastic Net (uses both L1 and L2 penalties) performed best for this data set, and then the value for the hyperparameter C that made it the best choice of penalty. The values and results appear in Table 5.

Table 5: Hyperparameter Tuning Values and Results for the Decision Tree Classifier Model

Regularization Type (Penalty)	Hyperparameter	Tested Value/Methods	Best Value/Method
Lasso (L1)	C	-4, 3, 10	0.0001
Ridge (L2)	C	-4, 3, 10	-
Elastic Net	C	-4, 3, 10	-
	L1 Ratio	0.25, 0.50, 0.75	-

Seeing as the grid search identified Lasso regression as the most effective regularization type, the grid with these values was fitted to the training data and yielded a best cross-validation score of approximately 0.757 alongside a runtime of 1 minute.

Hyperparameter Tuning

In the pursuit of improved individual multi class model accuracies, the results of the initial grid searches were taken as indicators as to where to continue exploring parameter values. Thus, a second round of hyperparameter tuning took place prior to the ensembling with the input parameter values centered around the outputs, these initial optimal values, of the initial grid searches. Inputs and results for each parameter grid are presented in Tables 6 and 7 below. In the case of the AdaBoost classifier, it is of utmost importance we remark that the limitation for the accuracy appeared to arise from the fact we had set decision stumps as the algorithm, therefore, we explored extending the maximum depth in hope of overcoming said impedance. Adding to that, the logistic regression model was not selected for this second round of tuning as it had an

accuracy low enough with a runtime high enough to render it impractical seeing as these signs pointed to improvements in accuracy coming from refinement of the other models.

Table 6: Results of Second Grid Search for Random Forest Classifier

Hyperparameter	Tested Value/Methods	Best Value/Method
Number of Estimators	1100, 1150, 1200 1250	1250
Maximum Tree Depth	None	None
Minimum Samples to Split	2, 3	2
Minimum Samples per Leaf	1, 2, 3	2
Maximum Features Considered	sqrt, log2, 0.5	0.5

As opposed to the initial grid search, we let the depth be unconstrained while the other parameters followed the aforementioned logic to determine their inputs. With this new combination of parameter values, the best CV accuracy remained at 0.674 with a runtime of 7 minutes.

Table 7: Results of Second Grid Search for AdaBoost Classifier

Hyperparameter	Tested Value/Methods	Best Value
Learning Rate	0.25, 0.275, 0.3	0.3
Number of Estimators	1050, 1100, 1200, 1250	1250
Maximum Depth	2, 3	3
Minimum Samples per Leaf	4, 5, 6	5

This improved combination of hyperparameter values saw minimal increases in the best cv accuracy, with the optimized one being approximately 0.556 alongside a runtime of 10 minutes and 35 seconds.

Accounting for the satisfactory accuracy results of the binary classifier models, these were immediately proceeded with and went directly for the ensembling.

Due to the high runtime of tuning the Logistic Regression model we opted not to further tune it with tighter bounds as we judged it would be costly.

Ensembling

As a final step, the models used for each of the selected targets were ensemble into a single object, one object for the rating group classification and one for the investment grade classification, and tested against the untouched testing data set. For this ensemble, we employed Voting Classifier instances to be able to tune the reliance and contributions of the ensemble object on each of the models through exploration of the weights, keeping the goal of maximizing performance in terms of accuracy. This final optimization step was expected to yield more desirable, reliable, and applicable results.

<u>Multi-Classification Ensemble</u>

Since the models used for this sort of classification are more complex and presented a wide range of non-ideal accuracies, we accounted for five overarching possibilities: three where one of the models was the most dominant, one where two of the models were the most dominant, and the case where the initially designed dominance was not enough and the edge cases where a model is even more dominant than initially expected. For the inputs we used combinations of weights between the three models that we thought best reflected the dominance at hand.

Table 10: Multiple Classification Model Ensembled Object GridSearch Inputs and Output

Dominant Model	Individual Model Weight			Best Voter Weight Distribution
	Random Forest Weight	AdaBoost Weight	LR Weight	
Balanced Weights	1	1	1	
	2	1	1	
	1	2	1	

	1	1	2	(5,1,1)
Random Forest	3	1	1	
	4	1	1	
AdaBoost	1	3	1	
	1	4	1	
Logistic Regression Dominant	1	1	3	
	1	1	4	
Combined Bias Towards 2 Models	2	2	1	
	2	1	2	
	1	2	2	
Edge Case Extremes	5	1	1	
	1	5	1	
	1	1	5	

Using the (5, 1, 1) weight distribution, meaning the extreme edge case where the random forest absolutely dominated the ensembled object, the ensembled object was tested against the testing set alongside each of the individual models and yielded the best CV score among them: approximately 0.651. We also trained a second ensemble of only the Random Forrest and ADABoost models which yielded a CV accuracy of 0.675 with weights (1,1), even though it yielded a higher score than the triple ensemble we were still convinced that the presence of the different nature of Logistic Regression would be beneficial when generalizing.

Given our beliefs we ran the test set on both the ensembles yield test accuracies of 0.7324 for the triple ensembling and a test accuracy of 0.7265 for the Random Forest and ADABoost ensembling, which confirmed our belief of the benefit of including Logistic Regression to the ensemble.

Binary Classification Ensemble

In the same manner as before, different combinations of weights for the models within the ensembled binary object were tested with the overarching categories being similar or identical with the major difference being the balanced input is solely (1,1) and the edge cases only reaching up to a max weight of 4. All inputs and finds for this final hyperparameter tune are detailed in Table 11.

Table 11: Binary Classification Model Ensembled Object GridSearch Inputs and Output

Decision Tree Classifier Weight	Logistic Regression Classifier Weight	Best Voter Weight Distributions
1	1	(1, 1)
2	1	
1	2	
3	1	
1	3	
4	1	
1	4	

Once again, using the ideal weight distribution (1, 1), the ensembled object alongside the individual binary classifiers were tested against the untouched testing set and the ensembled test object yielded a tied score with the individual decision tree, both tied at a testing accuracy of 0.8176.

Conclusions

General Conclusions

As expected from the selection of our multi-class models, the combination of the triple-ensemble model did heighten the balance, reliability, and blatant accuracy as the natures of the models contrasted effectively and therefore strengthened the ensembled object. In other words, their tendencies in the bias-variance scale balanced each other out and resulted in a heightened testing accuracy as seen before. Therefore, we take this generalizable model, which is able to effectively use credit score data to identify where a client will lie in the overarching Moody Scale categories, to be a success.

Regarding the binary classifier model, unexpectedly, the combination of the models provided no improvement on the testing accuracy and therefore rendered the ensembled object to be as effective as the decision tree by itself. Despite expectation of logistic regression helping to counter the decision trees tendency to overfit, to balance out the variance that the tree brings to the table with bias of its own, we see that for the case of credit score features the threshold for what is considered noise is higher than for other sets. In other words, what usually causes decision trees to overfit (excessive depth), actually pairs well with the high dimension count of a credit score data set when it comes to classifying under a binary target.

Limitations and Possible Amendments

Our triple-ensemble model benefited greatly from the inclusion of the Logistic Regression model into it, although we opted not to do a second more tight Grid Search to further tune its hyperparameters, out of concern for the long run time and low accuracy independently. With that in mind it's safe to assume that our model could benefit from more tuning of the hyperparameters for Logistic Regression. In addition to that it could also have proven beneficial to extend the dataset, either with new or repeated observations, to better cover the minority classes even though we took actions to try and ensure the model was able to capture them.

References

Textbooks

1. Raschka, Sebastian, Yuxi Hayden Liu, and Vahid Mirjalili. Machine Learning with PyTorch and Scikit-Learn. 4th Edition. Packt Publishing, 2022.
2. Bowles, Michael. Machine Learning in Python. Wiley, 2015.
3. Hackeling, Gavin. Mastering Machine Learning with Scikit-Learn. 2nd Edition. Packt Publishing, 2017.
4. Alpaydin, Ethem. Introduction to Machine Learning. The MIT Press, 2014.
5. Moncecchi, Guillermo, and Raúl Garreta. Learning Scikit-Learn: Machine Learning in Python. Packt Publishing, 2013.

Google Colab Notebook
https://colab.research.google.com/drive/1ay__sQu6BX33WOY4wfRVk5Vo8ja9ZIcq?usp=sharing

Github Link
https://github.com/gabrielgreisen/Credit-Risk-Modeling-with-Machine-Learning.git